



第六章 AT89系列单片机的内部资源及应用

刘剑锋

ljf@csu.edu.cn





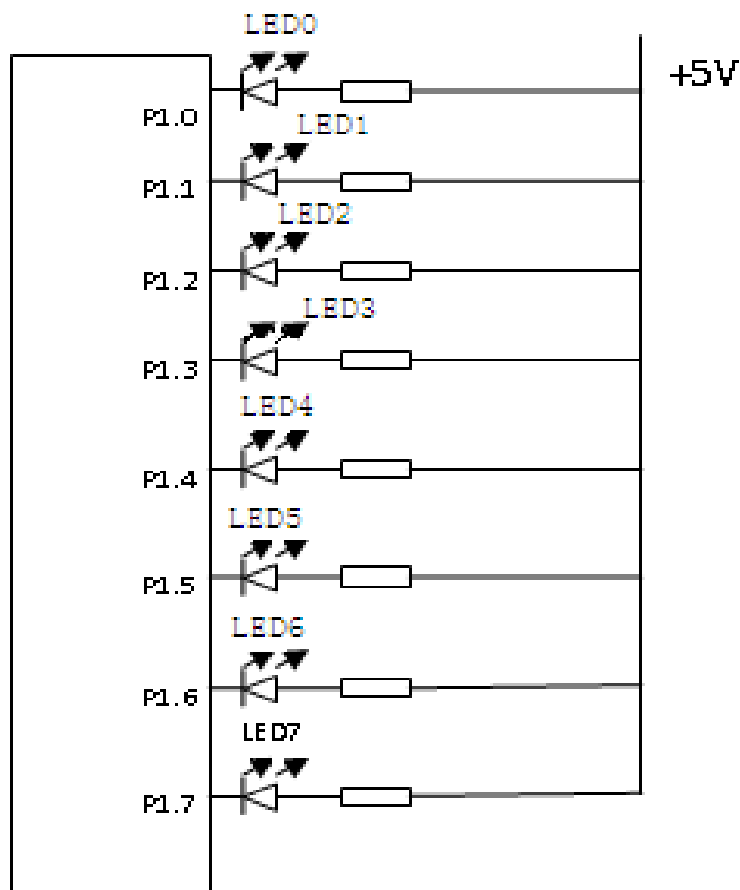
主要内容

- 6.1 AT89系列单片机的并行口及其应用
- 6.2 AT89系列单片机的中断系统
- 6.3 AT89系列单片机定时/计数器
- 6.4 AT89系列单片机的串行接口及串行通信

§ 6.1 AT89系列单片机的并行口及其应用

1. 指示灯

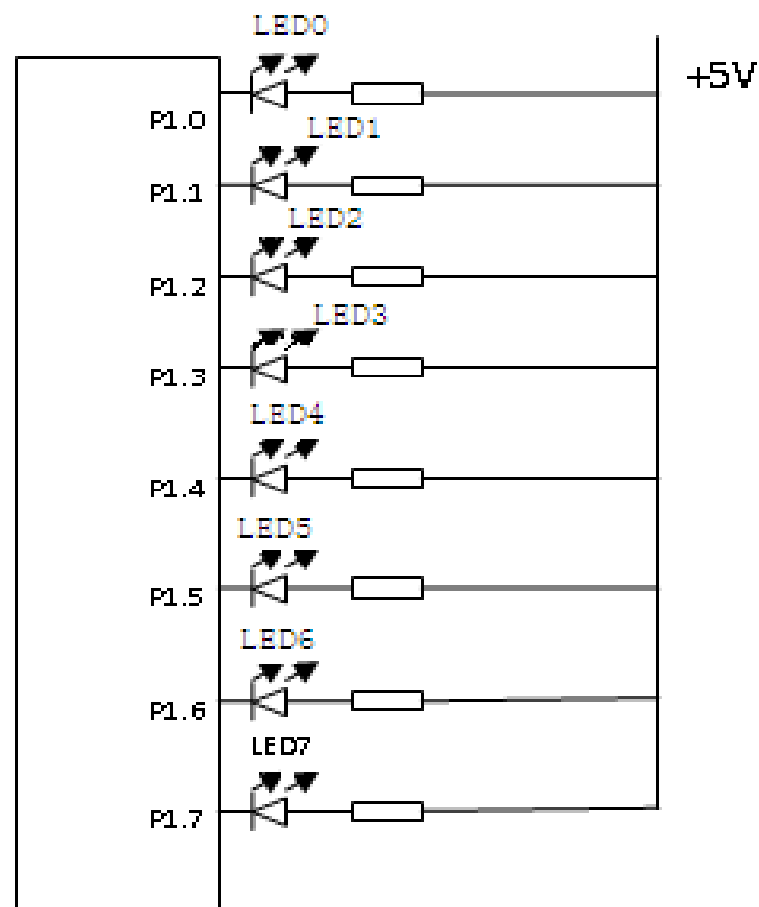
例6-1 用51单片机的并行口P1口驱动8个发光二极管，使8个发光二极管从上到下轮流点亮。



输出低电平0时，LED有电流通过发光，
输出高电平1时，无电流通过不发光

汇编语言如下：

```
                MOV A, #0FEH  
AGAIN:MOV P1, A  
                LCALL DELAY  
                RL A  
                SJMP AGAIN  
DELAY:MOV R7, #20  
                MOV R6, #250  
                DJNZ R6, $  
                DJNZ R7, DELAY
```



C51语言程序如下:

(1) 查表法

```
#include<reg52.h>
```

```
Void main( )
```

```
{
```

```
    unsigned char LED[8]={0xfe, 0xfd, 0xfb, 0xf7, 0xef, 0xdf, 0xbf, 0x7f}
```

```
    unsigned char i;
```

```
    unsigned int j;
```

```
    while{1}
```

```
    { for(i=0;i<8;i++)
```

```
        {P1=LED[i];
```

```
        for(j=0;j<30000;j++);    //延时
```

```
        if ( i==8 ) i=0xff;      //为了下一次循环LED灯从上到下点亮，置
```

```
ffH
```

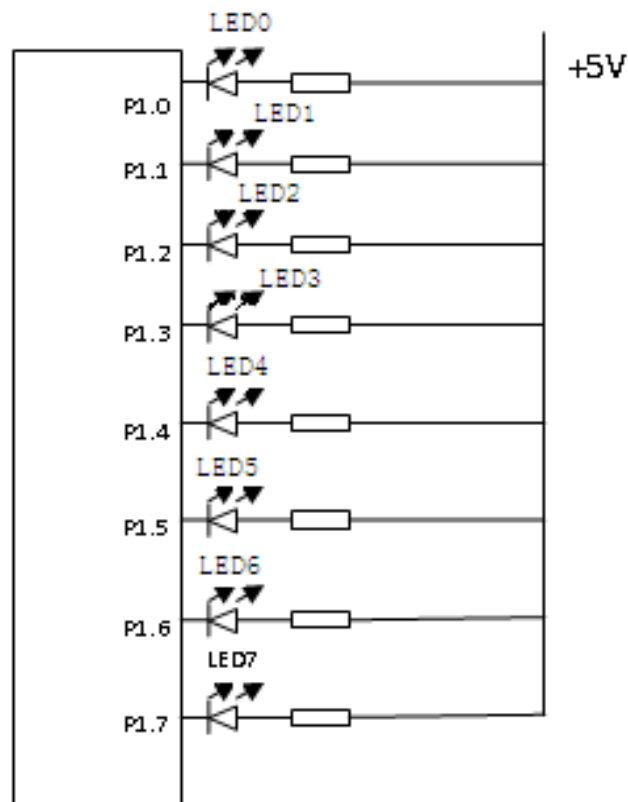
```
    }
```

```
}
```

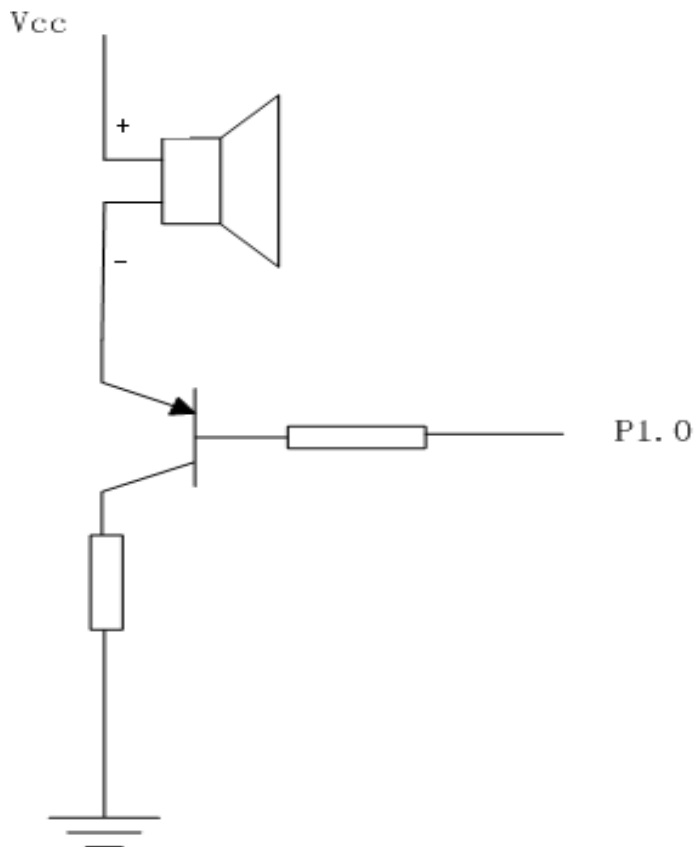
```
}
```

(2) 移位法

```
#include<reg52.h>
#include<intrins.c>
void main( )
{
    unsigned char i,a=0xfe;
    unsigned int j;
    while{1}
    { P1=a;
      a=_crol_(a,1);
      //调用循环左移函数_crol_, 变量a循环左移
      1位
      for(j=0;j<30000;j++);    //延时
    }
}
```



2. 蜂鸣器



- 当P1.0引脚输出0时，三极管导通，在蜂鸣器两端加工作电压，蜂鸣器发出声音；
- 当P1.0引脚输出1时，三极管截止，蜂鸣器不发声。

例6-2 编程使蜂鸣器响铃10次，每次响铃发出250Hz的声音，持续时间为0.5s，静音1s，单片机使用12MHz的晶振。实现该功能的程序如下：

```
MOV R3, #10
AA1: CLR P1.0
      MOV R1, #250
AA2: LCALL DELAY2 ; 调用延时2ms子程序
      CPL P1.0
      DJNZ R1, AA2      ; 250Hz响铃0.5s

      SETB P1.0
      MOV R2, #2
AA3: MOV R1, #250
AA4: LCALL DELAY2
      DJNZ R1, AA4
      DJNZ R2, AA3      ; 静音1s
      DJNZ R3, AA1
      RET
```

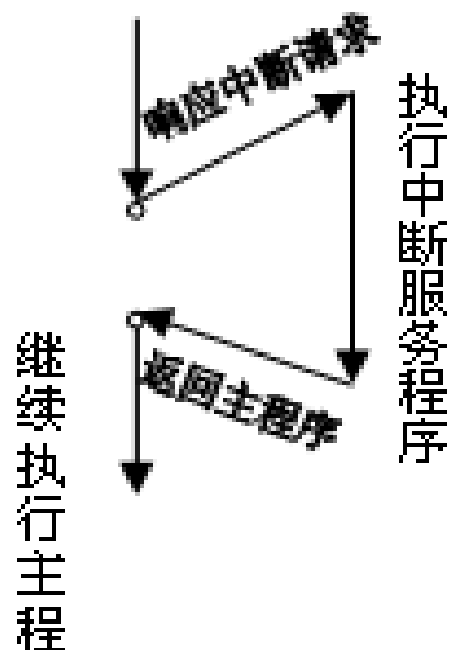
```
DELAY2: MOV R7, #10 ; 延时2ms程序

      AA5: MOV R6, #100
           DJNZ R6, $ ; 双周期指令，执行一次耗时2μs
           DJNZ R7, AA5
      RET
```

6.2 AT89系列单片机的中断系统

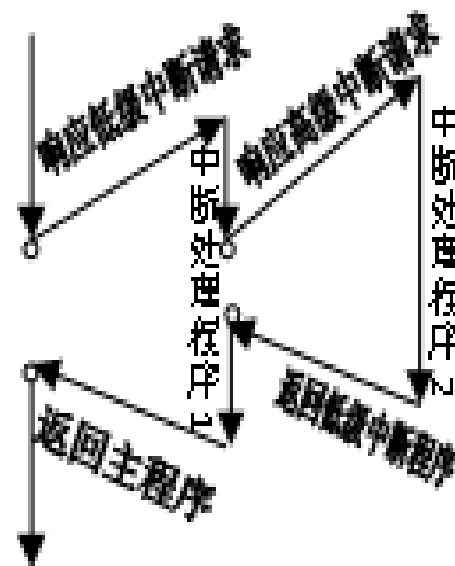
6.2.1 中断的基本概念

CPU 执行主程序



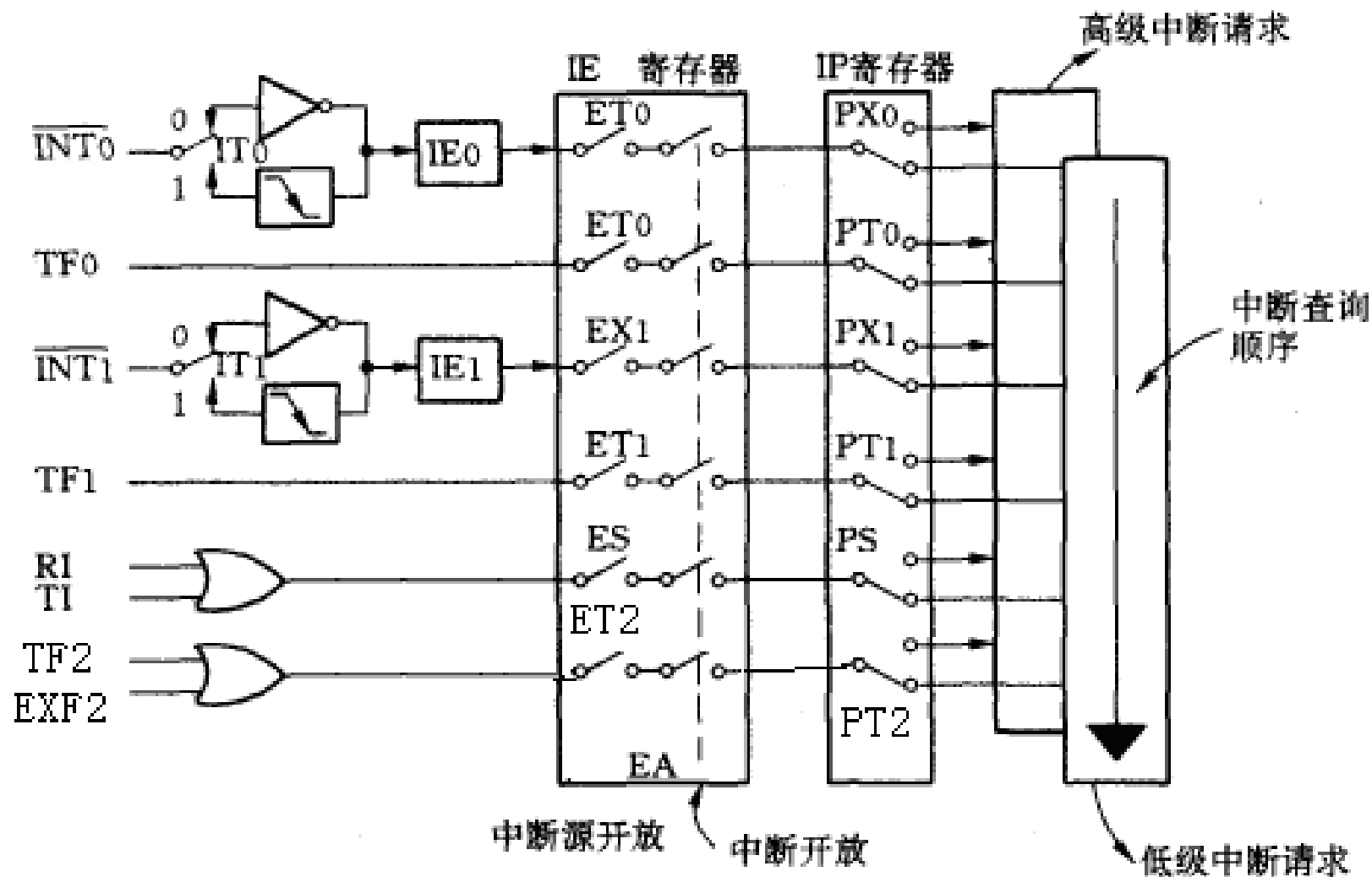
中断响应过程

CPU 执行主程序



二级中断嵌套过程

6.2.2 AT89系列单片机的中断系统



89S52中断系统结构

1. 89X52中断源

89X52有6个中断源，分别为两个外部事情中断请求源 $\overline{\text{INT1}}$ $\overline{\text{INT0}}$

和4个内部中断源分别为：

- 定时计数器T0、T1、T2 计数溢出事件中断请求
- 串行口发送或者接受完一个字节数据中断请求源。

89S51有5个中断源，没有T2计数溢出中断。

(1) 外部中断源

外中断0和外中断1的中断标志位和它们的触发方式控制位在特殊功能寄存器定时器控制寄存器（TCON）中的低4位。

D7	D6	D5	D4	D3	D2	D1	D0
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

定时器控制寄存器TCON

IT0 外部中断0触发方式控制位。

IT0=0，外部中断0触发方式选择为电平触发方式。

IT0=1，外部中断0触发方式选择为边沿触发方式。

IE0是外中断0的中断请求标志位。

当外部中断源（P3.2）引脚上有有效的中断请求信号，则置位IE0，向CPU请求中断，当CPU响应该中断时由硬件清“0”IE0。

IT1 外部中断1触发方式选择位。

IT1=0，外中断1为电平触发方式，

IT1=1，外中断1为边沿触发方式。

IE1 外中断1的中断请求标志位。

IE1=1，外中断1向CPU请求中断，CPU响应中断请求后，由硬件清“0”IE1位。

(2) 内部中断源

TF0 定时计数器T0计数溢出中断标志位。

T0允许计数后，从计数初值开始加1计数，当计数计满后（计数器所有位均为“1”），再加1，则计数溢出，此时由硬件自动置“1” TF0，向CPU请求中断，一直保持到CPU响应该中断时才由内部硬件清“0”。

TF1 定时计数器T1计数溢出中断标志位。

T1允许计数后，从计数初值开始加1计数，当计数溢出时，硬件自动置“1” TF1，向CPU请求中断，一直到CPU响应中断时由硬件清“0”。

89X52内部中断源定时计数器T2。

定时计数器T2中断：定时计数T2的计数溢出标志位TF2为“1”，或者T2的外部中断标志位EXF2为“1”，作为有效的中断请求，向CPU请求中断。CPU响应中断时不能自动清0，必须由软件清0。

串行口中断

串行口的接受中断标志RI（SCON.0）和发送中断标志TI（SCON.1）逻辑或以后作为内部的一个中断源。当串行口发送缓冲器发送完一个字符数据后，由硬件自动把发送中断标志位TI置“1”，向CPU请求中断。

需要注意的是，CPU响应中断后，发送中断标志位TI不会自动清“0”，必须由用户在中断处理程序中用软件清“0”

2. 中断控制

中断允许寄存器IE

D7	D6	D5	D4	D3	D2	D1	D0
EA	——	ET2	ES	ET1	EX1	ET0	EX0

EA AT89S系列单片机的CPU中断允许控制位。

EX0 外中断0的中断允许位。

EX1 外中断1的中断允许位。

ET1 定时器T1的中断允许位。

ES 串行口中断允许位。

ET2 定时器T2的中断允许位。

1 中断允许；0，中断屏蔽。

中断优先级寄存器IP

D7	D6	D5	D4	D3	D2	D1	D0
——	——	PT2	PS	PT1	PX1	PT0	PX0

- PX0 外部中断0的中断优先级控制位。
- PT0 定时计数器T0中断优先级控制位。
- PX1 外部中断1的中断优先级控制位。
- PT1 定时计数器T1中断优先级控制位。
- PS 串行口中断优先级控制位。
- PT2 定时计数器T2中断优先级控制位。

1 定义为高优先级; 0 定义为低优先级

在同一级别的中断源请求源中，中断优先顺序为：





中断源	入口地址
外部中断0	0003H
定时器T0	000BH
外部中断1	0013H
定时器T1	001BH
串行口中断	0023H
定时器T2	002BH

6.2.3 外部事件中断及应用

1. 外部事件中断源的初始化

1) 设置外部事件中断请求信号的触发方式。

如果外部中断触发方式采用电平触发方式，IT0或者IT1位清0，

如果用边沿触发方式，IT0或者IT1位置“1”。

2) 开放CPU中断允许位：SETB EA。

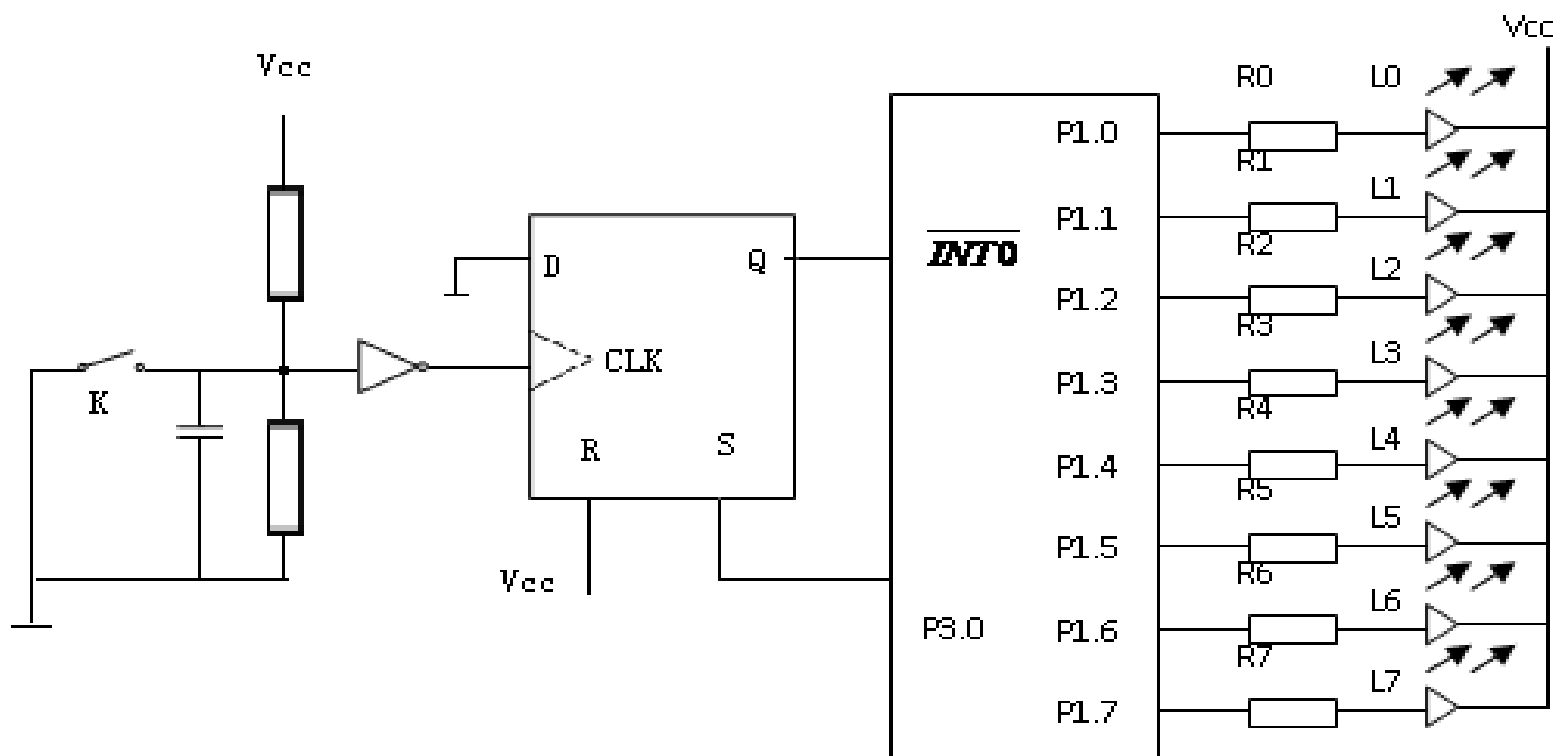
3) 设置外部事件中断允许控制位：SETB EX0 或者SETB EX1。

4) 设置中断源中断优先级。

2. 外部中断应用举例

例6-3 如图6-9所示，P1.0~P1.7为输出线，外接指示灯L0~L7，采用外部中断0电平触发方式改变指示灯L0~L7的显示状态。

正常显示时，灯L0~L7自上而下逐一点亮，当有外部中断请求时，灯L0~L7全部点亮并闪烁显示10次。闪烁完成后，继续从暂停的位置接着逐个点亮灯的操作。



主程序:

ORG 0000H

LJMP MAIN

ORG 0003H

LJMP INTO

ORG 0030H

MAIN:MOV SP, #70H ; 设置堆栈区

CLR IT0 ; 电平触发方式

SETB EA ; 开放CPU中断允许

SETB EX0 ; 设置外中断0中断允许

CLR PX0

MOV A, #0FEH

DISPLAY:MOV P1, A

ACALL DELAY ; 延时

RL A

AJMP DISPLAY

DELAY:MOV R7, #200 ; 延时子程序

DELO: MOV R6, #100

DJNZ R6, \$

DJNZ R7, DELO

RET

中断服务程序:

ORG 3000H

INT0:PUSH ACC ; 保护现场

PUSH PSW

CLR P3.0 ;

MOV R5, #10

REPEAT:MOV A, #00H ; 全部点亮

MOV P1, A

ACALL DELAY

MOV A, #0FFH ; 全部熄灭

MOV P1, A

ACALL DELAY

DJNZ R5, REPEAT ; 闪烁10次

POP PSW

POP ACC

RETI

END

C51程序:

```
#include <reg52.h>
#include <intrins.h>
void delay ( );
sbit P3_0=P3^0;
unsigned char a;
main ( )
{
    a=0xfe;
    IT0=0;    //外中断0电平触发方式
    EA=1;
    EX0=1;
    PX0=0;
    while (1)
    {
        P1=a;
        a=_crol_(a,1);
        //左移, 产生下一个显示控制码
        delay ( );
    };
}
```

```
void delay( )
```

```
{
    unsigned int b;
    b=20000;
    while (b>0) b--;
}
```

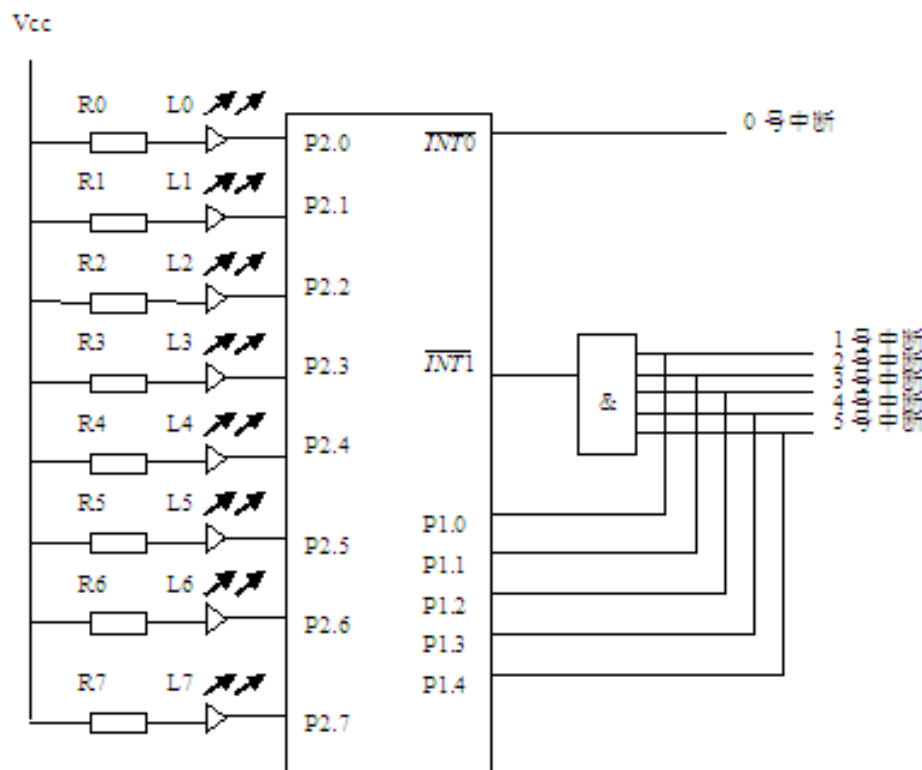
```
extern0 ( ) interrupt 0 using 1    //中断函数
```

```
{
    unsigned char i;
    P3_0=0;
    for (i=10;i>0;i--)            //闪烁10次
    {
        P1=0x00;
        delay( );
        P1=0xff;
        delay( );
    }
}
```

例6-4 外部中断源的扩展。

如图6-10中，共有6个外部中断源，分别为0号、1号、2号、3号、4号、5号中断源，中断请求采用电平触发方式，0号中断源的中断优先级别最高，单片机优先响应，单片机接受到0号中断源中断请求时，8个LED灯闪烁显示10次，1号、2号、3号、4号、5号中断源为低级中断源，当有其中一个发出中断请求信号时，则触发外部中断的中断服务程序中，通过查询P1.0~P1.4的状态，判定是哪一个中断请求，然后执行响应的中断服务程序，

查询的顺序，决定这几个中断源中断优先级，先查询的优先级别高，后查询的优先级别低，本例中低优先级中断源的中断优先循序为5号、4号、3号、2号、1号。



主程序:

```
ORG 0000H
LJMP MAIN
ORG 0003H                ; 外中断0中断入口地址
LJMP PINT0
ORG 0013H                ; 外中断1中断入口地址
LJMP PINT1
ORG 0030H
MAIN:CLR IT0              ; 设置外中断0为电平触发方式
      CLR IT1              ; 设置外中断1为电平触发方式
      MOV IE, #85H        ; 设置CPU中断允许, 外中断0、外中断1中断允许
      MOV IP, #01H        ; 设置外中断0为高级, 外中断1为低级
LOOP: .....
      LJMP LOOP
```

INT0的中断服务程序:

PINT0: PUSH ACC

PUSH PSW

SETB RS1

; 把工作寄存器区切换到2区, 中断处理程序中使用2区R0~R7

MOV R1, #10 ; 闪烁显示10次

FLASH: CLR A

MOV P1, A ; P1口灯全亮

ACALL DELAY

CPL A

MOV P1, A ; P1口灯全熄灭

ACALL DELAY

DJNZ R1, FLASH

POP PSW

POP ACC

RETI

INT1的中断服务程序:

PINT1:CLR EA ; 关中断

PUSH ACC

PUSH PSW ; 保护现场

SETB EA ; 开中断

JNB P1.4 PINT1_5

JNB P1.3 PINT1_4

JNB P1.2 PINT1_3

JNB P1.1 PINT1_2

JNB P1.0 PINT1_1

RETURN:CLR EA ; 关中断

POP PSW

POP ACC ; 恢复现场

SETB EA ; 开中断

RETI

PINT1_5: ; 5号中断服务程序

AJMP RETURN

PINT1_4:..... ; 4号中断服务程序

AJMP RETURN

PINT1_3:..... ; 3号中断服务程序

AJMP RETURN

PINT1_2:..... ; 2号中断服务程序

AJMP RETURN

PINT1_1:..... ; 1号中断服务程序

AJMP RETURN

C51程序如下:

```
#include <reg52.h>

void delay ( );           //延时函数声明

void PINT1_5();           //5号中断调用的函数声明
void PINT1_4();           //4号中断调用的函数声明
void PINT1_3();           //3号中断调用的函数声明
void PINT1_2();           //2号中断调用的函数声明
void PINT1_1();           //1号中断调用的函数声明

sbit P1_0=P1^0;
sbit P1_1=P1^1;
sbit P1_2=P1^2;
sbit P1_3=P1^3;
sbit P1_4=P1^4;
unsigned char a;
```

```
main ( )
{
    IT0=0; //外中断0电平触发方式
    IE=0x85;
    IP=0x01;
    ...
    while (1);
}
```

```
void delay( )
{
    unsigned int b;
    b=20000;
    while (b>0) b--;
}
```

void PINT1_5() { }	//5号中断调用的函数定义
void PINT1_4() { }	//4号中断调用的函数
void PINT1_3() { }	//3号中断调用的函数
void PINT1_2() { }	//2号中断调用的函数
void PINT1_1() { }	//1号中断调用的函数


```
extern0 ( ) interrupt 0 using 2      //0号中断函数
{
    unsigned char i;
    for (i=10;i>0;i--)
    {
        P1=0x00;
        delay( );
        P1=0xff;
        delay( );
    }
}
```

```
extern1 ( ) interrupt 1      //外部中断1中断函数
{
    if (P1_4= =0) PINT1_5();      //1-5号中断为同级中断，按照查询顺序只响应
    一个

    else if (P1_3= =0) PINT1_4();

        else if (P1_2= =0) PINT1_3();

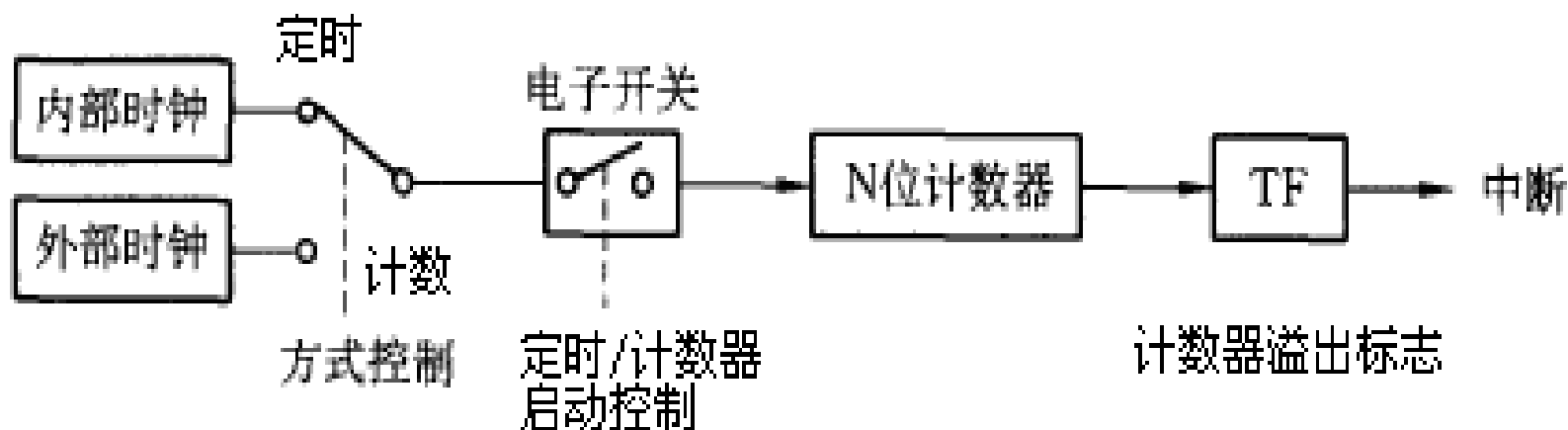
            else if (P1_1= =0) PINT1_2();

                else if (P1_0= =0) PINT1_1();

}
```

6.3 AT89系列单片机定时/计数器

6.3.1 定时/计数器的一般结构和工作原理



定时/计数器结构图

1. 定时方式

从计数器中计数初值 a 开始加1计数，直至计数溢出所占用的时间为：

$$t = T * (2^N - a)$$

例如：某单片机应用系统，使用12MHz的晶振，定时/计数器为16位的定时计数器，计数初值为15536，则该定时/计数器定时时间多长。

单片机的机器周期T为：

$$T = 12 * \frac{1}{f_{osc}} = 12 * \frac{1}{12 * 10^6} = 1\mu s$$

定时时间t为：

$$t = T * (2^N - a) = 1\mu s * (2^{16} - 15536) = 50ms$$

2. 计数方式

计数方式是对外部输入的脉冲信号计数，外部输入的脉冲信号从特定的引脚上输入，计数器对脉冲信号加1计数，即信号发生从1到0的负跳变，计数器自动加1。

6.3.2 定时/计数器T0、T1功能和使用方法

有关的特殊功能寄存器有以下几个：TH0、TL0、TH1、TL1、TMOD、TCON, 中断相关的寄存器还有IE、IP

1. 方式寄存器TMOD

D7	D6	D5	D4	D3	D2	D1	D0
GATE	$C/\overline{T}$	M1	M0	GATE	$C/\overline{T}$	M1	M0

低4位D0~D3 用于控制定时/计数器T0;

高4位D4~D7 用于控制定时/计数器T1。

(1) M1M0工作方式选择位

M1	M0	功能说明
0	0	方式0，13位定时/计数方式
0	1	方式1，16位定时/计数方式
1	0	方式2，8位自动重装初值定时/计数方式
1	1	方式3，T0分为两个独立的8位定时/计数器；T1停止计数

(2) $C/\overline{T}$ 位 定时/计数模式选择

0为定时方式

1为计数方式

(3) GATE门控位

GATE=1时，定时计数器开始计数， $\overline{INT0}$ 或者 $\overline{INT1}$ 上必须为高电平；

GATE=0时，是否启动定时/计数器开始计数，不受外部引脚输入电平的控制

2. 控制寄存器TCON

D7	D6	D5	D4	D3	D2	D1	D0
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

TR0为定时/计数器T0的运行控制位；**TR1** 为 **T1**运行控制位

当GATE=0时，TR0为1时启动T0计数开始，TR0为0时停止T0计数

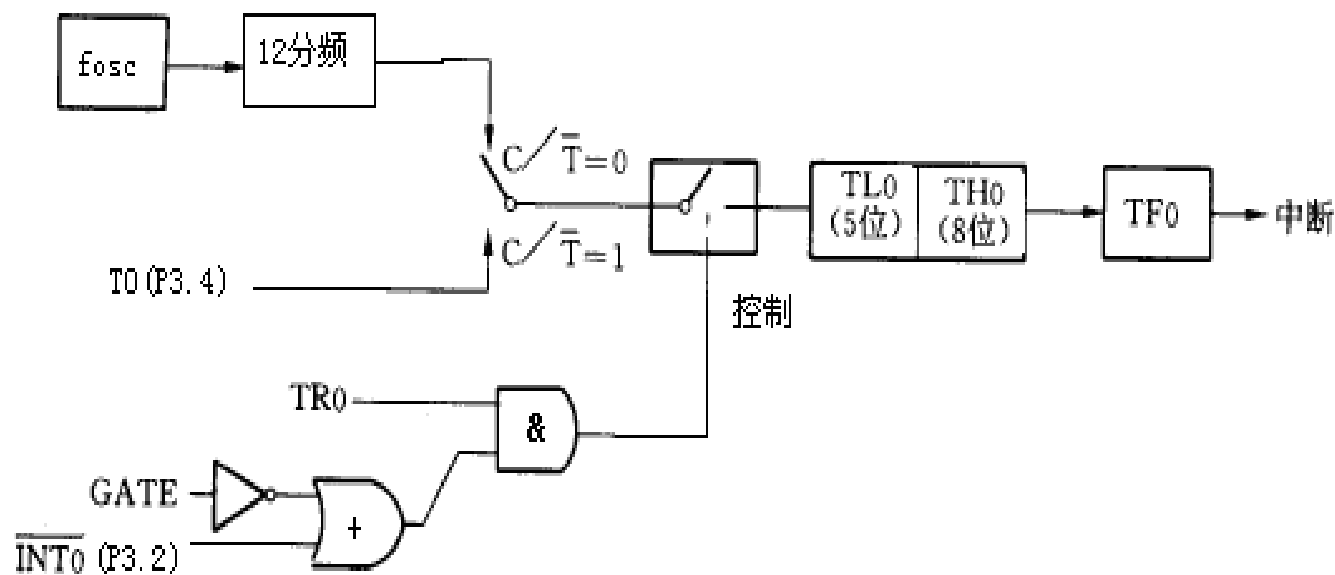
当GATE=1时， $\overline{\text{INT0}}$ 引脚为高电平且TR0为1时启动T0计数开始；
 $\overline{\text{INT1}}$ 引脚为高电平且TR1为1时启动T1计数。

TF0 T0 溢出标志位

TF1 **T1** 溢出标志位

3. T0、T1的工作方式

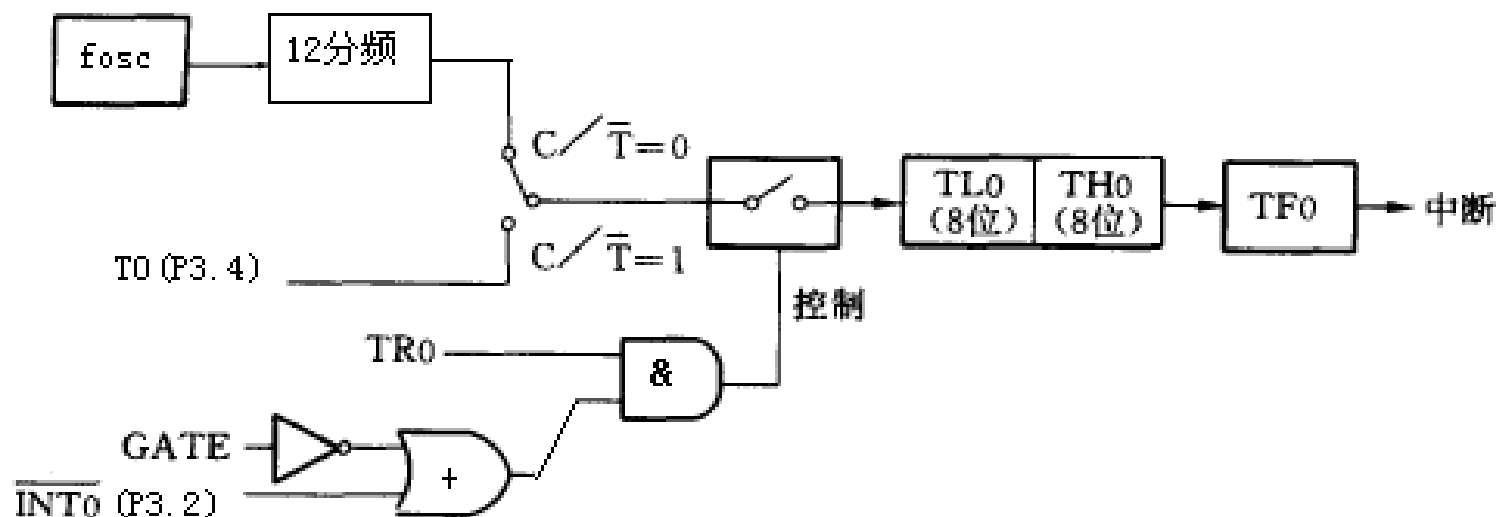
(1) 方式0 13位的定时计数方式



定时时间 $t = \frac{1}{f_{osc}} * 12 * (2^{13} - a) \text{ s}$ a ——计数初值

➤ 计数器的计数范围为: $1 \sim 2^{13}$

(2) 方式1 16位的定时计数方式



定时时间 $t = \frac{1}{f_{osc}} * 12 * (2^{16} - a) \text{ s}$

计数器的计数范围为：1~65536

例6-6 设 $f=12\text{MHz}$, 定时/计数器T0工作于方式1, 产生50ms定时, 试分别采用中断和查询的方式编程实现P1.0引脚产生周期为1s的方波。

要使得P1.0引脚产生周期为1s的方波, 则P1.0引脚高低电平持续的时间应精确为500ms, 本例使用定时/计数器T0精确定时50ms, 则10次50ms中断时定时时间为500ms。

中断方式:

在T0中断服务程序中设置一个计数单元, 计数初值为10每次50ms中断时候, 计数单元数值减1, 当计数单元数值减为0时, 恰好是10次中断, 即时间为500ms, 500ms到来时引脚P1.0取反。

查询方式:

在主程序中不断查询TF0的状态, 当TF0为1时, 表示50ms定时时间已到, 在主程序中判断是否10次50ms定时已到, 如是则P1.0引脚取反, 若否则继续循环等待。用查询方式, 每次TF0置1后, TF0必须由软件清0。

计数初值：
$$t = \frac{1}{f_{osc}} * 12 * (2^{16} - a) \quad s$$

$$a = 2^{16} - \frac{f_{osc}}{12} * t$$

$a=15536=3CB0H$ （十六进制）

即计数器中TL0的初值为 B0H，TH0的初值为 3C H

TMOD定时/计数器寄存器：

定时/计数器T0工作于方式1，故M1M0=01；工作于定时工作方式，

$C/\overline{T} = 0$ ；GATE=0，

则TMOD=01H

① 中断方式

设置定时/计数器工作于方式1, 定时方式, 同时中断允许。

主程序:

```
ORG 0000H
LJMP MAIN
ORG 000BH
LJMP INTT0
MAIN:MOV SP, #60H
MOV TH0, #3CH
MOV TL0, #0B0H
MOV TMOD, #1
MOV IE, #82H    ; 允许T0向CPU申请中断
SETB TR0        ; 启动T0开始计数
MOV 30H, #10    ; T0中断次数计数单元初始化为10
SJMP $
```

T0中断服务程序:

```
INTT0:MOV TH0,#3CH
        MOV TL0,#0B0H
        DJNZ 30H,RETT0
        MOV 30H,#10
        CPL P1.0      ; P1.0取反
RETT0:RETI
```

C51程序如下:

```
#include <reg52.h>
sbit P1_0=P1^0;
unsigned char a=10;
main ( )
{
    TH0=0x3c;
    TL0=0xb0;
    TMOD=0x01;
    IE=0X82;
    TR0=1;
    while (1);
}
```

```
timer0 ( ) interrupt 1 using 1
{
    TH0=0x3c;           //重赋计数初值
    TL0=0xb0;
    a--;
    if (a==0)
    {
        a=10;
        P1_0=~P1_0;
    }
}
```

②查询方式

采用查询方式，此时T0仍然工作方式1的定时方式，但是，设置为不允许T0向CPU申请中断。程序如下：

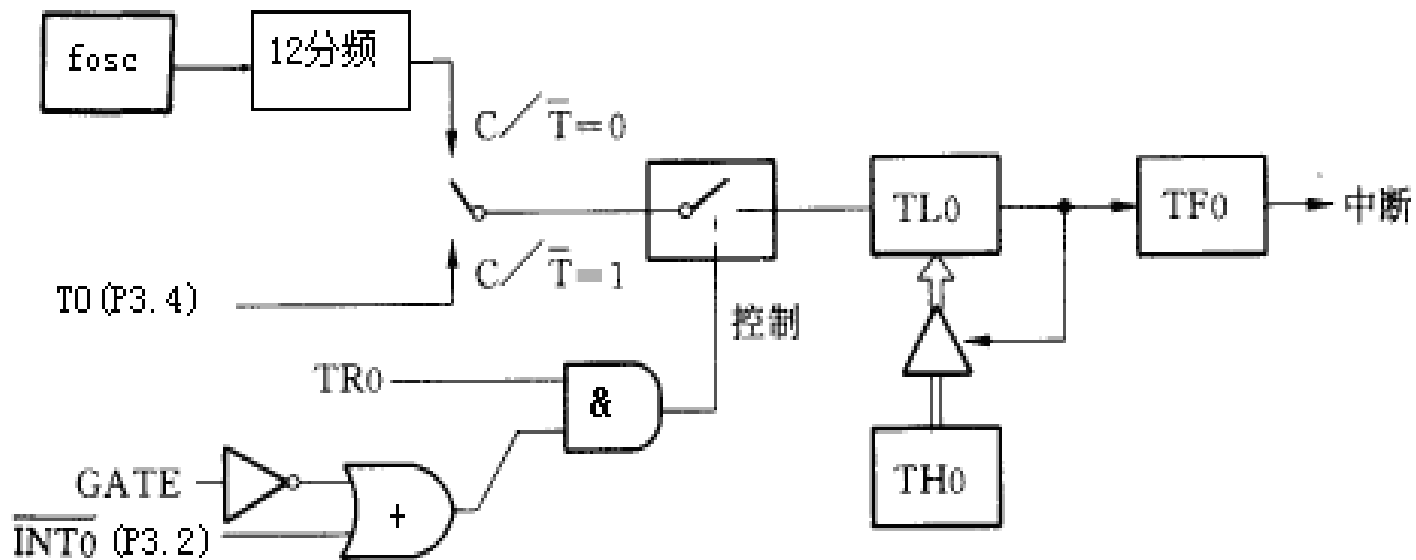
```
ORG 0000H
MOV SP, #60H
MOV TH0, #3CH
MOV TL0, #0B0H
MOV TMOD, #1
MOV IE, #00H      ; 关闭中断允许
MOV 30H, #10
SETB TR0
WAIT: JNB TF0, $    ; TF0=0循环等待，TF0=1继续往下执行
      CLR TF0      ; TF0清0，以便下次定时50ms的查询
      MOV TH0, #3CH ; 50ms定时的计数初值再次赋给定时/计数器T0
      MOV TL0, #0B0H
      DJNZ 30H, WAIT ; 计数单元减1不为0，继续等待
      MOV 30H, #    ; 计数单元减1为0，则再次把计数初值赋给计数
      单元
      CPL P1.0      ; P1.0取反
      LJMP WAIT     ; 返回WAIT，继续查询等待
```

C51程序如下:

```
#include <reg52.h>
sbit P1_0=P1^0;
unsigned char a;
main ( )
{
    TH0=0x3c;
    TL0=0xb0;
    TMOD=0x01;
    IE=0;
    TR0=1;
    while(1)
```

```
{
    for(a=10;a>0;a--)
    {
        while (TF0==0);
        TF0=0;
        TH0=0x3c;
        TL0=0xb0;
    }
    P1_0=~P1_0;
}
}
```

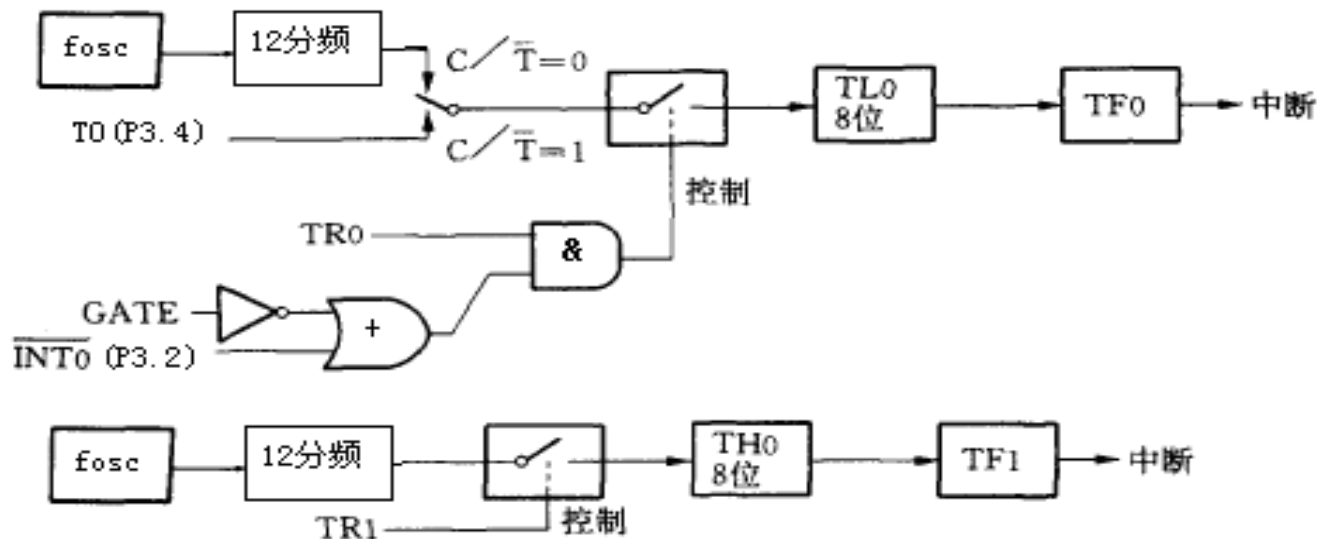
(3) 方式2：8位自动重装初值定时/计数方式



定时/计数器T0方式2逻辑结构

$$t = \frac{1}{f_{osc}} * 12 * (2^8 - a) \quad s$$

(4) 方式3



T0分为两个独立的8位计数器TL0和TH0，

TL0计数器使用定时/计数器T0的所有控制位和状态标志位：GATE、

$\overline{INT0}$ 引脚（P3.2）、T0（P3.4）、TR0、TF0，

TL0作为一个独立的8位定时器或者外部事件计数器，

TL0计数溢出时置“1”溢出标志TF0，同时，TL0计数器清0，

TL0的计数初值必须由软件赋值。

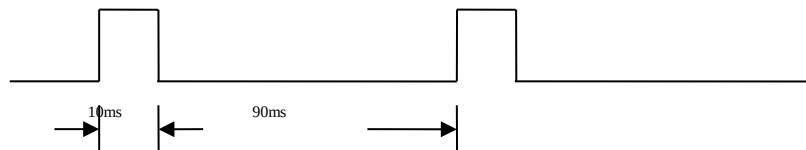


TH0只能工作于8位的定时方式，并借用定时/计数器T1的控制位和状态标志位TR1和TF1。

TR1=1时，定时/计数器TH0开始计数，当TH0计数溢出时，溢出标志TF1置“1”。

6.3.3 定时/计数器的初始化编程及应用

例6-8 占空比不同信号发生器



P1.0引脚输出周期为100ms 周期信号，高电平10ms，低电平90ms

使用定时/计数器T0产生10ms定时，T0工作于方式1，则计数初值如下：

$$t = 1\mu s * (2^{16} - a) = 10ms$$

$$a = 65536 - 10000 = 55536 = D8F0H \text{ (十六进制)}$$

具体程序如下：

```
ORG 0000H
LJMP MAIN
ORG 000BH
LJMP PINT0
ORG 0030H
MAIN:MOV SP, #60H
MOV 31H, #9
MOV TMOD, #01
MOV TH0, #0D8H
MOV TL0, #0 F0H
MOV IE, #81H
CLR P1.0
CLR 00H
SETB TR0
HERE:AJMP HERE
PINT0:PUSH ACC
PUSH PSW
MOV TH0, #0D8H
MOV TL0, #0 F0H
JB 00H, LOW
SETB P1.0
SETB 00H
SJMP OUT
```

； 设置一个计数器
； 定时/计数器初始化

```
LOW:CLR P1.0
      DJNZ 31H, OUT
      MOV 31H, #9
      CLR 00H
OUT:POP PSW
     POP ACC
     RETI
```

； 若00H为1， 则输出90ms

例6-9 秒表的程序设计

单片机使用12MHz的晶振，定时/计数器T0工作于方式2，产生250 μ s定时，每1s使秒表显示缓冲器30H~32H实时计时。

缓冲器分配如下：

30H高四位为小时的十位，低四位为小时的个位

31H高四位为分钟的十位，低四位为分钟的个位；

32H高四位为秒的十位，低四位为秒的个位。

每1秒钟秒加1计时，当计时到59秒，下1秒到来时为1分钟，秒清0，分钟加1；

当计时到59分59秒，下1秒到来时为1小时，分钟、秒清0，小时加1

能够计时的最长计时时间为99小时59分59秒。

解：（1）确定TMOD

定时/计数器T0工作于方式2的定时方式的定时方式

D7	D6	D5	D4	D3	D2	D1	D0
GATE	$C/\overline{T}$	M1	M0	GATE	$C/\overline{T}$	M1	M0
*	*	*	*	0	0	1	0

（2）计算计数初值

由定时时间的计算公式：

$$t = \frac{1}{f_{osc}} * 12 * (2^8 - a) \quad s$$

计数初值a=6

T0中断4000次时时间为1s

(3) 程序如下:

ORG 0000H

LJMP MAIN ; 设置跳转到主程序

ORG 000BH ; T0中断服务程序入口地址

LJMP PINTT0 ; 设置跳转到中断服务程序

ORG 0030H

MAIN: MOV SP, #60H ; 主程序

MOV 36H, #10H

MOV 37H, #0A0H ; 4000= (0FA0H) 中断次数放入计数单元

MOV TMOD, #02H

MOV TL0, #6

MOV TH0, #6

MOV IE, #82H

SETB TR0

SJMP \$

PINTT0:PUSH PSW

; T0中断服务程序

PUSH ACC

SETB RS0

; 选择1区的工作寄存器

DJNZ 37H, RETURN

DJNZ 36H, RETURN ;4000次中断未到中断返回

MOV 36H, #10H

MOV 37H, #0A0H ; 4000次中断到来, 计数器单元恢复初始值

MOV R0, #32H

MOV A, @R0

ADD A, #1

DA A ; 调整为十进制相加

MOV @R0, A

CJNE A, #60H, RETURN ;修改秒表计时值, 秒单元加1, 不为60秒中断返回

MOV @R0, #0 ; 为60秒, 秒单元清0, 分钟单元加1

DEC R0

MOV A, @R0



ADD A, #1

DA A

MOV @R0, A

CJNE A, #60H, RETURN ;若为60分钟, 分钟单元清0, 小时单元加1

MOV @R0, #0

DEC R0

MOV A, @R0

ADD A, #1

DA A

MOV @R0, A

RETURN: POP ACC ; 恢复现场

POP PSW

RETI ; 中断返回



C51程序如下：

```
#include <reg52.h>
```

```
unsigned int a=4000;
```

```
unsigned char time_buf[]={0, 0, 0};
```

```
main ( )
```

```
{
```

```
    TH0=0x06;
```

```
    TL0=0x06;
```

```
    TMOD=0x02;
```

```
    IE=0x82;
```

```
    TR0=1;
```

```
    while(1) { }
```

```
}
```



```
timer0 ( ) interrupt 1 using 1  //T0的中断函数
{
    a--;
    if (a==0)
    {
        a=4000;
        time_buf[2]+=1;           //秒单元加1
        if (time_buf[2]>=60)      //若为60秒
        {
            time_buf[2]=0;       //秒单元清0
            time_buf[1]+=1;       //分钟单元加1
            if (time_buf[1]>=60)   //若为60分
            {
                time_buf[1]=0;   //分钟单元清0
                time_buf[0]+=1;   //小时单元加1
            }
        }
    }
}
```

6.3.4 AT89S系列单片机看门狗定时器的编程方法

AT89S52 看门狗定时器由一个13位定时器及WDTRST（字节地址为6AH）寄存器构成。开启看门狗定时器后, 13位定时器会自动加1计数，如不对定时器复位则每计数8192，个机器周期溢出一次，并产生一个高电平复位信号，使单片机系统复位。

对于12MHZ的时钟脉冲每8192us（约8.192ms）产生一个复位信号，启动看门狗定时器，当系统超过8.192ms没有对看门狗定时器复位，看门狗定时器溢出，让系统复位，为了系统既能正常工作又不会出现死机（程序跑飞），大约在8ms内必须喂狗一次，即对看门狗定时器进行复位。

启动看门狗命令格式如下：

```
MOV    0A6H, #1EH
```

```
MOV    0A6H, #0E1H    ； 启动看门狗
```

对0A6H单元依次写入数据1EH和0E1H，激活看门狗监视定时器，如果程序正常执行，在看门狗监视定时器溢出时间前，再次复位看门狗监视定时器。

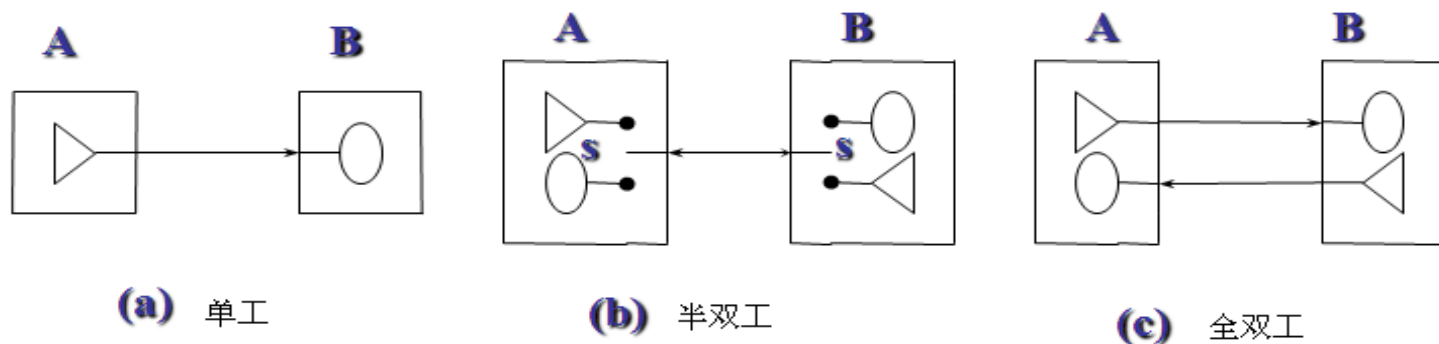
c51语言来完成此功能，程序如下：

```
#include <reg52.h>
sfr  WDTRST=0xa6;
main()
{
    .....;
    WDTRST=0x1e;    //启动看门狗
    WDTRST=0xe1;
    While(1)
    {
        WDTRST=0x1e; //喂狗
        WDTRST=0xe1;
        .....;
        .....;这部分执行时间必须少于8ms（对12MHZ时钟）
    }
}
```

6.4 AT89系列单片机的串行接口及串行通信

6.4.1 串行口的基本通信方式

51系列单片机提供一个全双工的异步串行接口



异步通信数据格式



波特率是指每秒钟内传输的二进制数的位数

例如：串行通信的波特率为 2400，表示每秒钟内传输 2400 位二进制数。

若字符格式为：1位数据位，8位数据位，无校验位，1位停止位，这样每秒钟内数据传输 240 个字符。

6.4.2单片机串行口及控制寄存器

- 串行口发送缓冲器和接收缓冲器SBUF
- 串行口控制寄存器SCON
- PCON中SMOD位，用来控制串行口的波特率

1. 串行口控制寄存器SCON

D7	D6	D5	D4	D3	D2	D1	D0
SM0	SM1	SM2	REN	TB8	RB8	TI	RI

SM0，SM1是串行口工作方式选择位

SM0	SM1	工作方式	功能说明	波特率
0	0	方式0	移位寄存器方式	
0	1	方式1	8位异步通信方式	可变
1	0	方式2	9位异步通信方式	
1	1	方式3	9位异步通信方式	可变

TI 串行口发送中断标志位。

当串行口发送完一帧数据，硬件自动将TI置1，并向CPU请求中断。

TI必须由软件清0。

RI 串行口接收中断标志位。

当串行口接收完一帧数据，硬件自动将RI置1，并向CPU请求中断。

RI必须由软件清0。

SM2 方式2和方式3的多机通信控制位

如SM2=1，接受到的第9位数据（RB8）为0时不置位RI，则接收到的数据丢失，只有接收到的第9位数据（RB8）为1时，才将接受到的前8位数据送入SBUF，并置位RI，产生中断请求。

当SM2=0时，不论第9位接收到的是0还是1，都将接收到前8位数据送入SBUF中，并将RI置1，产生中断请求。

REN 串行接受允许控制位。

REN=1时，允许串行口接受数据；

REN=0时，禁止串行口接受数据。

TB8

方式2和方式3时发送数据的第9位数据

RB8

方式2和方式3时接收到的第9位数据

2. 电源控制寄存器PCON

D7	D6	D5	D4	D3	D2	D1	D0
SMOD	——	——	——	GF1	GF0	PD	IDL

SMOD 波特率加倍选择位。

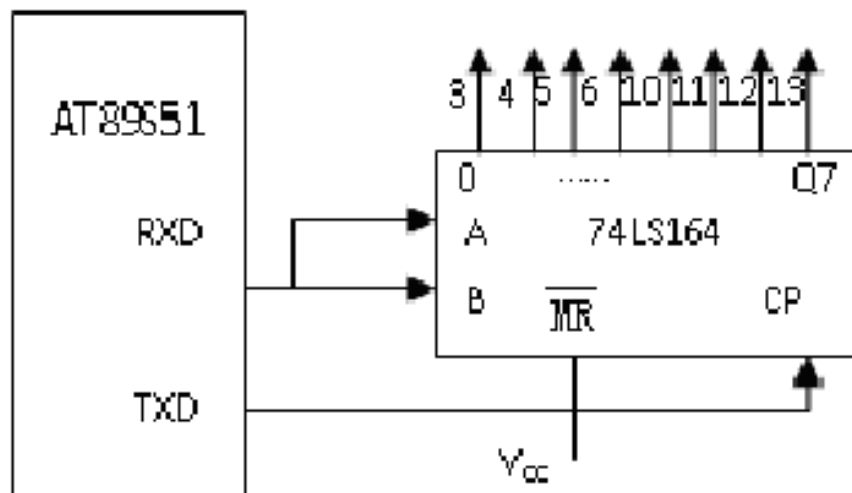
串行口工作在方式1、方式2、方式3时，如果SMOD=1，则串行口波特率加倍；
若SMOD=0，则波特率不会提高。

6.4.3 单片机串行通信工作方式

1. 方式0：同步移位寄存器工作方式

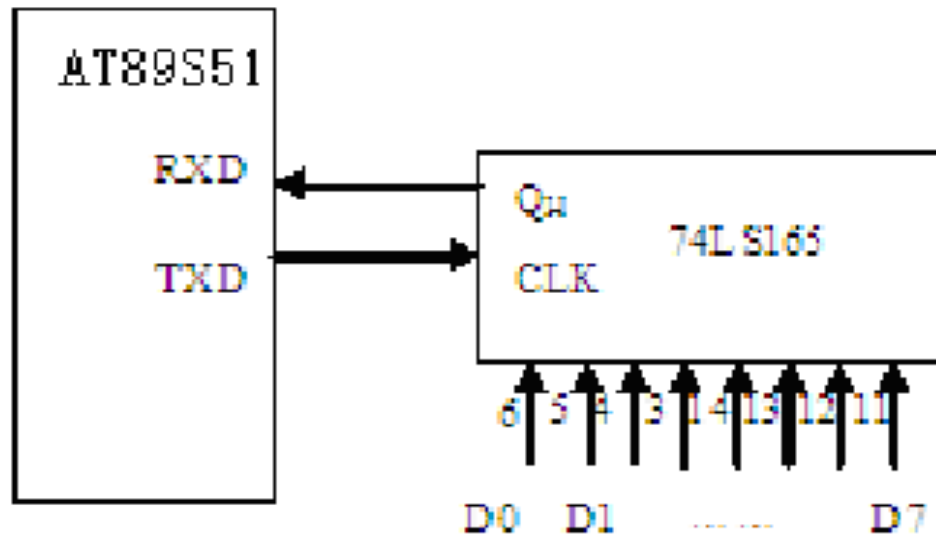
(1) 方式0发送

数据由RXD引脚串行输出，TXD引脚输出同步移位脉冲信号



输入/用于扩展并行输出口

(2) 方式0接收



2. 方式1

(1) 方式1发送

- 当TI=0时，执行指令MOV SBUF, A，CPU向串行口发送缓冲器SBUF写入一个字节的數據，启动串行口发送。
- 在TXD引脚上输出一帧信息，先发送起始位0，接着从低位开始输出8位数据，最后输出停止位1，发送后置位中断标志TI，输出完一个字符后串行口停止工作，CPU执行程序判断TI为1后软件清0TI中断标志，再向SBUF写入数据，启动串行口发送下一个字符。

(2) 方式1接收

- 接收到的数据装入接收数据缓冲器SBUF，置位RI，表示串行口接收到有效的一帧信息，向CPU请求中断。
- CPU响应中断时，取走SBUF接收数据缓冲器中已经接收的一个字节数据，如MOV A, SBUF，并软件清“0”RI接收中断标志位，如CLR RI。
- 接着串行口输入控制电路重新搜索RXD端上电平负跳变，接收下一个数据。

3. 方式2和方式3

(1) 方式2和方式3发送 类似于方式1

发送数据时，除了把一个字节数据写入SBUF发送缓冲器外，还需要把第9位数据写入SCON的TB8。

(2) 方式2和方式3接收 类似于方式1

不同的是：RB8中存放的是第9位数据

➤ 若SM2=1，串行口工作于多机通信方式：

- 当接收到第9位数据RB8=1时，数据有效接收，将数据分别装入SBUF和RB8中，置位RI；
- 当接收到第9位数据RB8=0时，数据丢失，并且不置位RI。

6. 4. 4单片机串行口的初始化编程及波特率设置

1. 串行口波特率

方式0 波特率 = 振荡频率/12

方式2 波特率= 2^{SMOD} *振荡频率/64

方式1和方式3的波特率：可变

➤ 波特率由定时计数器T1或T2的溢出率和SMOD位一起确定

1) T1为波特率发生器

方式1（方式3）波特率= 2^{SMOD} * (T1溢出率) /32

- 定时器T1作为波特率发生器时，禁止T1中断，T1工作于定时方式，一般选择工作方式2（8位自动重装初值工作方式），若计数初值为a，此时T1的溢出时间为：

$$t = 12 * \frac{1}{f_{osc}} * (2^8 - a)$$

T1的溢出率为溢出时间的倒数，此时波特率的计算公式为：

$$\text{方式1（方式3）波特率} = 2^{\text{SMOD}} * f_{osc} / (32 * 12 * (2^8 - a))$$

2. 串行口初始化编程

串行口初始化编程一般考虑如下几个方面：

- (1) 确定波特率
- (2) 确定串行口的工作方式
- (3) 确定串行口是否中断允许及其中断优先级

例如：已知振荡频率 $f_{osc} = 11.0592\text{MHz}$ ，对串行口初始化编程，
设置其工作于方式1发送/接收方式，波特率为9600，允许串行口中断。

解：① 波特率9600

使用定时计数器T1为波特率发生器，工作于方式2（8位自动重装初值工作方式）定时方式，计数初值为FDH，不允许T1中断。

②串行口工作于方式1，8位异步通信方式，则SCON=50H。

③允许串行口中断，则IE=90H。

初始化程序段如下：

MOV TMOD, #20H ; T1工作于方式2定时方式

MOV TH1, #0FDH

MOV TL1, #0FDH ; 设置计数初值

SETB TR1 ; 启动T1

MOV SCON, #50H ; 串行口工作于方式1，RI=0，TI=0

MOV IE, #90H ; 设置串行口中断允许

6.4.5 RS-232C串行口标准

常用的串行总线有RS-232C、RS-422、RS-485总线

1. RS-232C的电气特性及帧格式

RS-232C的电气标准为：-12V~-5V 为逻辑电平 1，

+5V~+12V 为逻辑电平

0。

通信距离最大为15米，传输速率为20kbps

2. RS-232C与TTL的电平转换

2组TTL电平到RS-232C电平转换器，

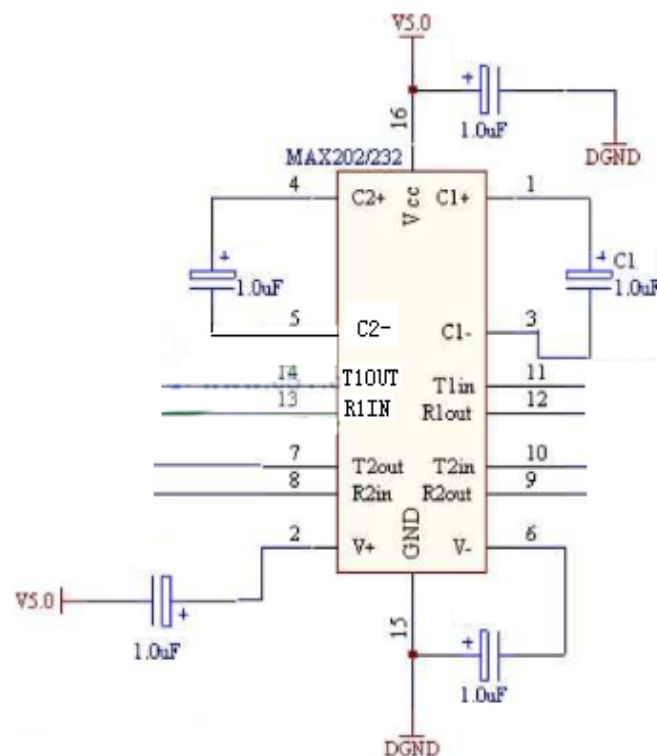
2组RS-232C电平到TTL电平转换器

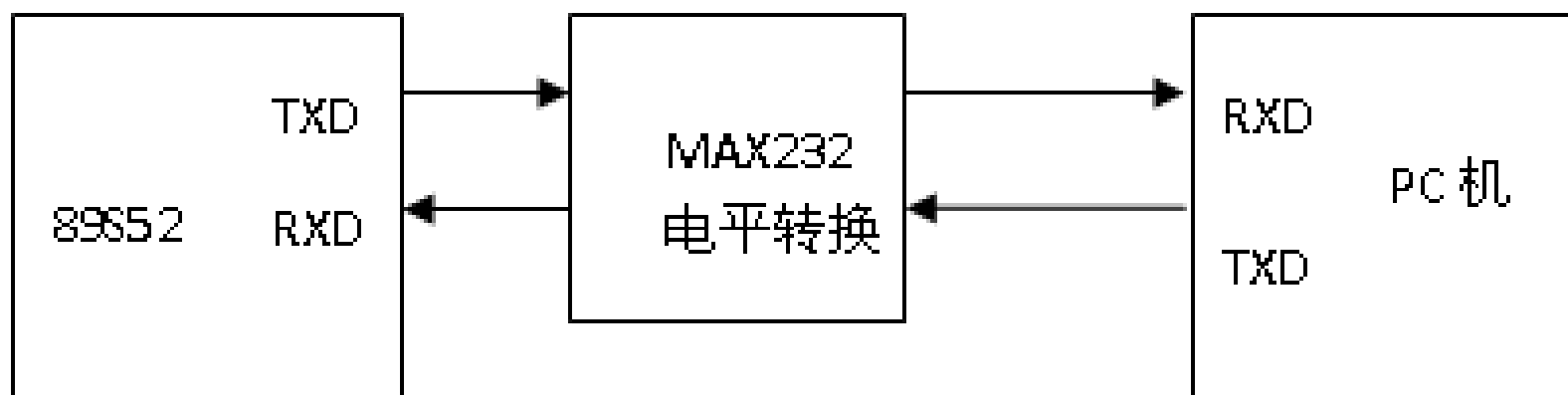
T1IN(TTL)----T1OUT (RS232)

T2IN(TTL)----T2OUT (RS232)

R1IN (RS232)---R1OUT (TTL)

R2IN (RS232)---R2OUT (TTL)





单片机和PC机的通信

6.4.6 RS-422、RS-485标准串行总线接口

1. RS-422标准接口

RS-422A的最大传输距离是300m，传输速度为10Mbps，在低速方式下，最大传输距离可达1200m。

RS-422接口电路如图6-30所示，

75174完成TTL电平到RS-422A电平转换，经过数据线传输，

75175把接收到的RS-422A电平转换为TTL电平输出。

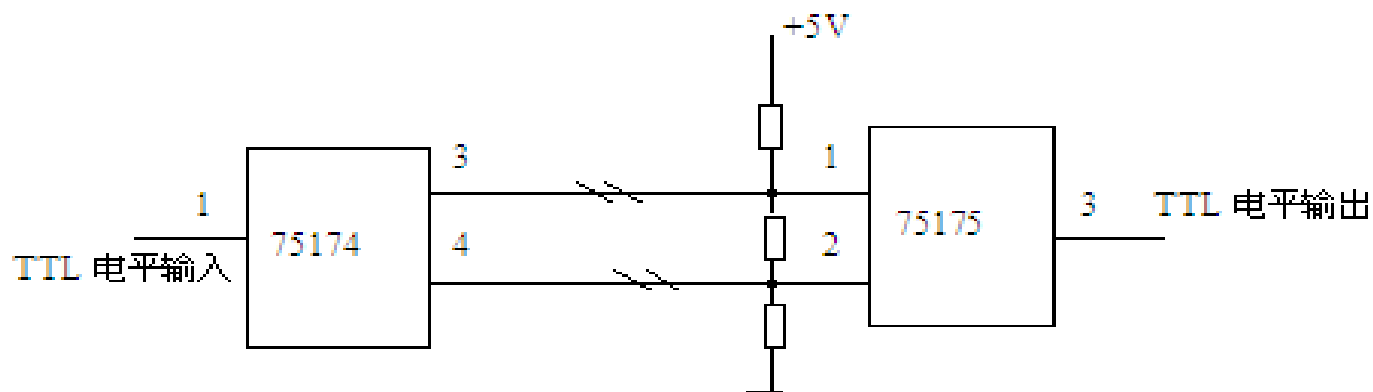


图6-30 RS-422接口电路

6.4.7 串行通信应用举例

1. 移位寄存器方式应用

例6-10 串行口方式0输出扩展并行口。

如图6-32所示，串行口外接两片74LS164，编程使得 0#74LS164的输出端并行输出存储单元30H单元数据，1#74LS164的输出端并行输出存储单元31H单元数据。

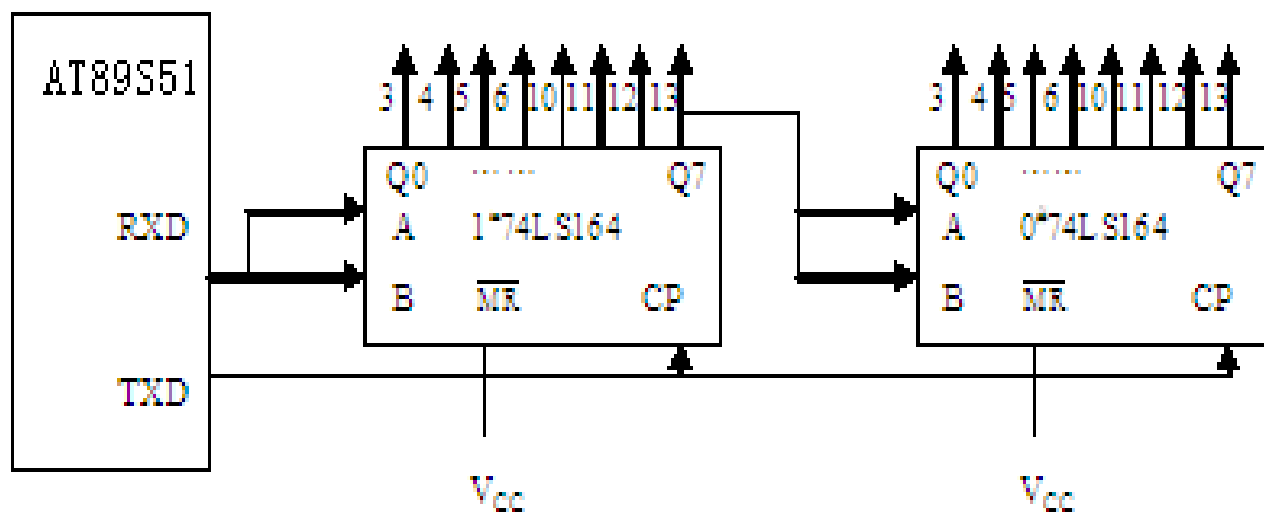


图6-32 串行口扩展16位并行输出电路

MOV SCON, #00H ;设置串行口工作于方式0

MOV R0, #30H

MOV R2, #02H

CIRCU:MOV A, @R0 ; 先发送30H单元数据

MOV SBUF, A ;数据送入串口发送缓冲

器, 启动串口发送

WAIT:JNB TI, WAIT ; 等待数据发送结束

CLR TI

INC R0

DJNZ R2, **CIRCU**

C51程序如下:

```
#include <reg52.h>
```

```
#define uchar unsigned char
```

```
#include <absacc.h>
```

```
#define data1 DBYTE[0x30]
```

```
#define data2 DBYTE[0x31]
```

```
void main (void)
```

```
{
```

```
    SCON=0;
```

```
    SBUF=data1;
```

```
    while(TI==0);
```

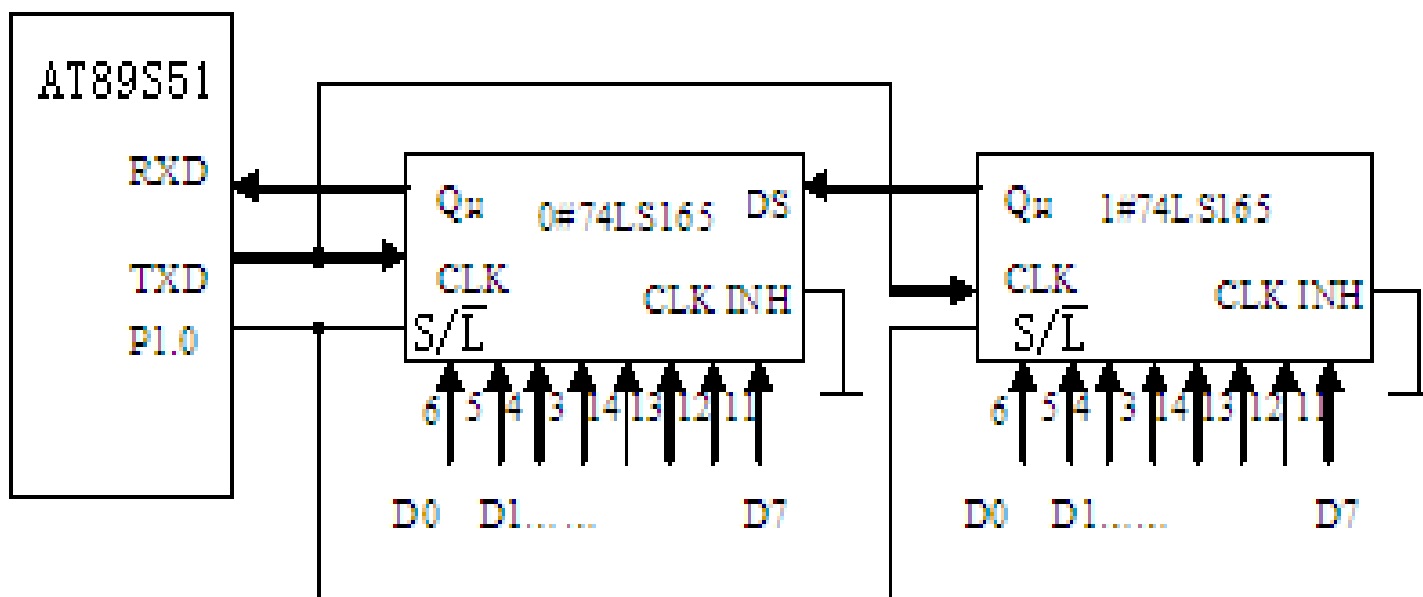
```
    TI=0;
```

```
    SBUF=data2;
```

```
}
```

例6-11 串行口方式0并行输入扩展。

如图6-33所示的16位接口电路读入数据，并把数据分别存放在30H和31H单元，其中0#74LS165的输入数据D0~D7存放于30H单元，对应D0存放在30H. 0位，D7存放在30H. 7位，1#74LS165的输入数据D0~D7存放于31H单元。



串行口扩展16位的并行输入电路



```
MOV R1, #02H          ; 接收字节数
MOV R0, #30H          ; 接收数据存储单元地址
CLR P1.0              ; 并行输入数据锁入移位寄存器
NOP
SETB P1.0             ; 允许移位寄存器移位工作
MOV SCON, #10H        ; 设置串行口工作于方式0，接收允
WAIT: JNB RI, WAIT     ; 查询RI位是否为1，等待接收数据
CLR RI                ; 清除接收标志，准备下次接收
MOV A, SBUF
MOV @R0, A            ; 存储接收数据
INC R0                ; 修改存储单元地址
DJNZ R1, WAIT         ; 两个字节数据是否接收完
RET
```


C51程序如下:

```
#include <reg52.h>
#define uchar unsigned char
uchar i,data1[];
sbit P1_0=P1^0;
void main (void)
{
    P1_0=0;
    nop();
    P1_0=1;
    SCON=0x10;
    for(i=0;i<2;i++)
    {
        while(RI==0);  RI=0;
        data1[i]=SBUF;
    }
}
```

2. 单机、多机通信应用

例6-12 两台单片机进行串行口通信，0#单片机发送数据，1#单片机接收数据，0#单片机将字符串‘AT89S52 Microcomputer’发送到1#单片机接收，并存储到其内部RAM从30H开始的存储单元。发送字符串以数据0结束，两个单片机的晶振频率均为11.0592MHz，波特率均为9600。
试编写两个单片机的串口程序。

解：0#单片机和1#单片机之间串行通信使用单工通信方式，一个单片机只发送数据，另外一个单片机只具有接受数据的功能。

(1) 两个单片机应该工作于方式1，8位波特率可变的异步通信方式。

0#单片机工作于发送方式，SCON=40H。

1#单片机工作于接收方式，SCON=50H。

(2) 使用定时/计数器T1作为波特率发生器，T1工作于方式2，8位自动重装初值的工作方式，波特率为9600时，查表可知计数初值应设定为FDH。

0#单片机的发送程序如下:

```
MOV TMOD, #20H      ; T1工作于方式2, 定时方式
MOV TH1, #0FDH       ; 计数初值为FDH
MOV TL1, #0FDH
MOV SCON, #40H       ; 串行口工作于方式1
SETB TR1             ; 启动定时/计数器T1
SETB TI              ; 为了便于用循环实现数据发送, 先置TI
MOV R4, #0           ; R4作字符串表指针
```

WAIT: JNB TI, WAIT

```
CLR TI
MOV DPTR, #TAB
MOV A, R4
MOVC A, @A+DPTR      ; 取字符
JZ RETURN            ; 字符串以0结束
MOV SBUF, A          ; 发送字符
INC R4               ; 为了取下一个字符, 修改字符串指针
LJMP WAIT            ; 跳转到等待查询, 是否发送完一个字符
```

RETURN: RET

TAB: DB 'AT89S52 Microcomputer' DB 0AH, 0DH, 0



C51程序如下:

```
#include <reg52.h>
unsigned char string[]={"AT89S52 Microcomputer \0"}; //字符串常量
unsigned char i=0;
main ( )
{
    TMOD=0X20;
    TH1=0XFD;
    TL1=0XFD;
    SCON=0X40;
    TR1=1;
    TI=1;
    while(string[i]!=0)
    {
        while(TI==0); TI=0;
        SBUF=string[i];
        i++;
    }
}
```

1#单片机的接收程序如下:

MOV TMOD, #20H ; T1工作于方式2, 定时方式

MOV TH1, #0FDH ; 计数初值为FDH

MOV TL1, #0FDH

MOV SCON, #50H ; 串行口工作于方式1, 接收允许REN=1

SETB TR1 ; 启动定时/计数器T1

MOV R0, #30H ; 设置存储单元的地址指针

WAIT: JBC RI, NEXT ; 循环检测RI是否为1, 为1, 接收SBUF中的数据

SJMP WAIT ; 不为1, 继续等待

NEXT: MOV A, SBUF ; 取SBUF中接收的一个字节的数据

MOV @R0, A ; 存放于存储单元

INC R0 ; 修改单元地址指针, 便于存放下

一个数据

LJMP WAIT

RET

C51程序如下：

```
#include <reg52.h>
unsigned char string[];           //字符串常量
unsigned char i=0;
main ( )
{
    TMOD=0X20;
    TH1=0XFD;
    TL1=0XFD;
    SCON=0X50;
    TR1=1;
    while(1)
    {
        while(RI==0);    RI=0;
        string[i]=SBUF;
        i++;
    }
}
```

例6-13 如图6-34所示，单片机系统使用11.0592MHz的晶体振荡器，串行通信的波特率为4800，0#单片机可以向1#单片机发送数据，1#单片机也可以向0#单片机发送数据，两个单片机之间可以进行全双工串行通信。

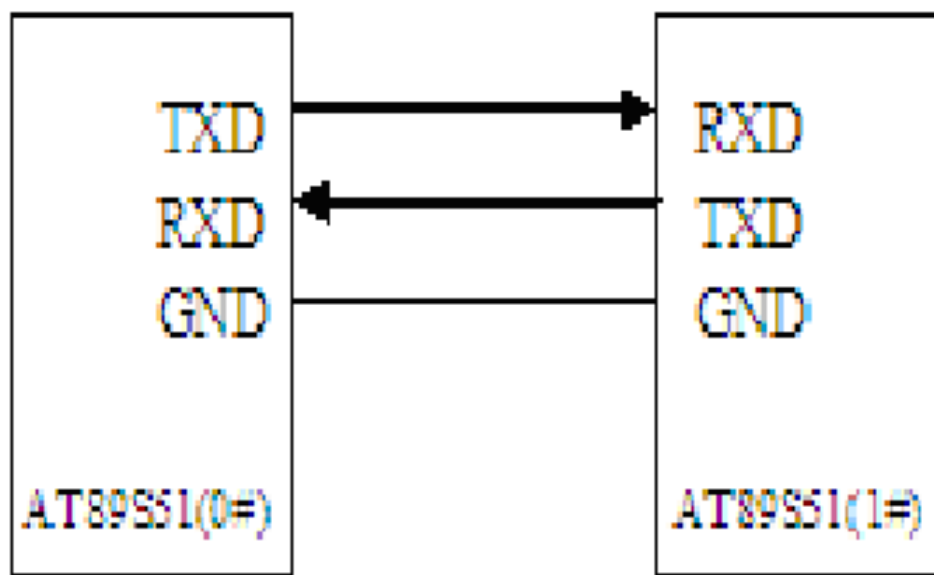


图6-34 2个单片机串行通信

解：（1）分析

根据程序设计要求，0#单片机和1#单片机地位相同，功能相似，任何一方都可以主动与对方通信，也可以相应对方的通信请求。

为了实现双工通信，定义2种通信协议：

① 0#单片机发送/1#单片机接收

0#单片机需要向1#单片机发送数据时，0#单片机先发出一个“接收数据请求”命令，1#单片机接收到命令后，如果准备好接收，则向0#单片机发送“接收准备就绪”命令，0#单片机接收到这个响应命令后，开始向对方发送数据，直到通信结束，两个单片机返回初始状态。

② 0#单片机接收/1#单片机发送

0#单片机需要从1#单片机接收数据时，0#单片机先发出一个“发送数据请求”命令，1#单片机接收到命令后，如果准备好发送，则向0#单片机发“发送准备就绪”命令，0#单片机接收到这个响应命令后，立即进入接收数据状态，直到通信结束，两个单片机返回初始状态。

程序中的几个命令，分别用一个字节的数据表示如下：

“接收数据请求”命令——01H

“接收准备就绪”命令——02H

“发送数据请求”命令——10H

“发送准备就绪”命令——20H

程序中设置两个标志位，位地址分别为00H和01H

00H 本机发送状态标志位。

若为 0，表示本机发送命令状态；若为 1，表示本机发送数据状态。

01H 本机接收状态标志位。

若为 0，表示本机接收命令状态；若为 1，表示本机接收数据状态。

(2) 程序

0#单片机的初始化程序：

```
ORG 0000H
```

```
LJMP MAIN
```

```
ORG 0023H
```

```
LJMP SINT
```

```
MAIN:ORG 0030H
```

```
MOV TMOD, #20H      ; T1工作于方式2，定时方式
```

```
MOV PCON, #00H      ; SMOD=0
```

```
MOV TH1, #0FAH      ; 波特率为4800时，T1的计数初值
```

```
MOV TL1, #0FAH
```

```
SETB TR1             ; 启动定时/计数器T1
```



```
MOV IE, #90H          ; 串行口中断允许
MOV SCON, #50H        ; 串行口工作方式1, 接收允许
CLR 00H               ; 设置本机为发送命令状态
CLR 01H               ; 设置本机为接收命令状态
MOV A, #01H           ; 发“接收数据请求”命令
MOV SBUF, A
SJMP $
```

当01H命令发送完后，0#单片机置位TI标志位，转入中断服务程序执行，中断服务程序如下：

```
SINT: JNB RI, SENT    ; 非接收中断, 转去发送处理
CLR RI
JB 01H, R_DATA        ; 位标志01H为1, 表示为接受数据
MOV A, SBUF           ; 接收命令
CJNE A, #01H, NEXT1   ; 非“接收数据请求”命令
```

MOV R6, #LEN_RCV ; 是“接收数据请求”命令，准备接收数据

MOV R1, #RCV_BUF ; 接收数据起始地址

SETB 01H ; 设置本机为接收数据状态

MOV A, #02H

MOV SBUF, A ; 发送“接收准备就绪”命令

LJMP RETURN

NEXT1: CJNE A, #10H, NEXT2 ; 非“发送数据请求”命令

MOV R7, #LEN_TRN

; 是“发送数据请求”命令，设置发送数据长度

MOV R0, #TRN_BUF ; 设置发送数据起始单元地址

SETB 01H ; 设置本机为发送数据状态

MOV A, # 20H

MOV SBUF, A ; 发送“发送准备就绪”命令

NEXT2: CJNE A, #02H, NEXT3 ; 对方“接收准备就绪”命令

MOV R7, #LEN_TRN

; 对方“接收准备就绪”命令，本机准备数据块发送，设置数据块长度

MOV R0, #TRN_BUF ; 设置数据块起始地址

SETB 00H	; 设置本机为发送数据状态
MOV A, @R0	; 取数据
MOV SBUF, A	; 发送数据
INC R0	; 修改单元地址
LJMP RETURN	; 跳转到中断返回
NEXT3: CJNE A, # 20H, NOCOM	; 对方“发送准备就绪”命令
MOV R6, #LEN_RCV	; 是对方“发送准备就绪”命令，本机准备接收数据
MOV R1, #RCV_BUF	; 设置接收数据起始地址
SETB 01H	; 设置本机为接收数据状态
NOCOM: LJMP RETURN	; 跳转到中断返回
R_DATA: MOV A, SBUF	; 取数据
MOV @R1, A	; 存储数据
INC R1	; 修改存储单元地址
DJNZ R6, RETURN	; 数据块是否接收完，R6中为数据块长度
CLR 01H	; 数据块接收完，置本机为接收命令状态
LJMP RETURN	
SENT: CLR TI	; 发送处理，清TI标志

JB 00H, T_DATA ; 若位标志为1，则为本机发送数据

AJMP RETURN ; 本机发送命令

T_DATA:MOV A, @R0 ; 取发送的数据

MOV SBUF, A ; 发送

INC R0 ; 修改发送单元地址

DJNZ R7, RETURN ; 数据块没发送完，直接中断返回，R7中为数据块长度

CLR 00H ; 数据块发送完，置本机为发送命令状态

RETURN:RETI

C51程序如下:

```
#include <reg52.h>
bit bit0, bit1;           //定义两个位标志
unsigned char a, i=0, j=0;
unsigned char rcv_buf[#LEN_RCV], trn_buf[#LEN_TRN];
    //定义接收数组和发送数组
main ( )
{
    TMOD=0X20;
    PCON=0;
    TH1=0XFA;
    TL1=0XFA;
    TR1=1;
    IE=0X90;
    SCON=0X50;
    bit0=bit1=0;
    SBUF=0X01;             //发“接收数据请求”命令
    while(1);
}
```

```
uart ( ) interrupt 4 using 1    //串口中断函数
{
    if (RI==1)                  //若为接收中断
    {
        RI=0;
        if(bit1==1) {          //判接收状态标志位，若为接收数据
            rcv_buf[j]=SBUF;    //接收数据
            j++;
        }
        if (j==#LEN_RCV) bit1=0;
    }

    else {                      //若为接收命令，判断命令的类型
        a=SBUF;
        if (a==1) {bit1=1;
            // 接收到的为“接收数据请求”命令
            SBUF=2; //发“接收准备就绪”命令
        }
    }
}
```

```
else if(a==0x10) { bit1=1; //接收到的为“发送数据请求”命令
                    SBUF=0X20;    //发“发送准备就绪”命令
                }

else if(a==2) {      //接收到的为“接收准备就绪”命令
    bit0=1;
    SBUF=trn_buf[i];    //1#接收准备就绪，0#发送数据
    i++;
}

else if(a==0x20) {    //接收到的为“发送准备就绪”命令
    bit1=1;    //使本机处于接收状态
}

}

else {SBUF=trn_buf[i]; //若为发送中断，发送数据
    i++;
    if (i==#LEN_TRN) bit0=0;
}

}
```


例6-14 多机通信程序举例。

其中一个为主单片机，其余的是从单片机，主从单片机的功能不同，这样的单片机应用系统为多机通信系统。

串行口的控制寄存器SCON中的SM2为多机通信控制位。

串行口以方式2或者方式3接收时，

若SM2=1时，

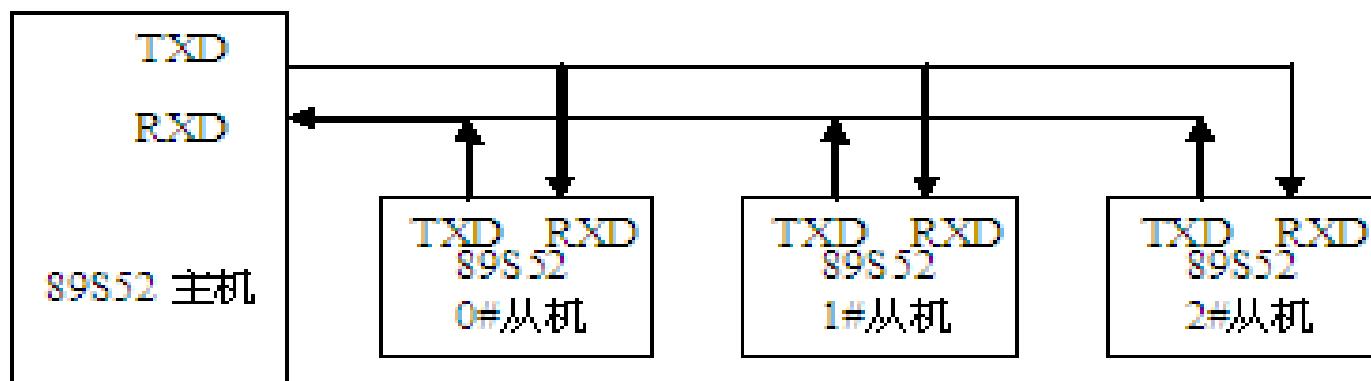
仅当接收到的第9位数据RB8为1时，数据才装入SBUF串口接收缓冲器，置位RI；

如果接收到的第9位数据RB8为0时，则不产生中断标志RI，数据丢失。

当SM2=0时

则接收到一个数据后，不管第9位数据RB8是何值，数据都装入SBUF，并置位RI。

➤ 多机通信中就是应用单片机这个特性，实现主从式多机通信。





主从机通信的方式如下：

(1) 每个从机分配一个地址，占用一个字节，三个从机的地址分别为0、1、2，各个从机的初始化程序将串行口编程为9位异步通信方式（方式2或者方式3），置位多机通信标志**SM2=1**，允许从机响应串行口接收中断。

(2) 主机能够向特定的从机发送数据。主机给特定的某一从机发送数据，此时其它从机不接收该数据，主机发送数据给从机时，先发送1个字节的地址，以辨认与之通信的从机，主机发送地址信息的第9位数据为1，地址信息所有的从机都能接收，主机发送的命令或数据信息的第9位数据为0，只有地址信息与主机发送的地址信息一致的从机，才允许接收数据。

(3) 从机接收到主机发送的地址信息，与自身的地址比较，如果一致，则SM2=0，如果不一致，仍然保持 SM2 = 1。

(4) 从机接收主机发送的命令，命令信息第9位数据为0，则SM2=0的从机正常接收该命令，SM2=1的从机丢弃该命令。这样保证只有选中的从机才能有效接收命令。

(5) 主机发送的命令为要求从机接收数据或者要求从机发送数据两种，根据命令情况从机作相应的处理程序：接收数据或者发送数据。

(6) 从机接收完主机数据或向主机发送完数据后，重新置位SM2=1，以便下次通信能够顺利实现。

如图所示的多机通信系统中，系统使用11.0592MHz的晶振，波特率为2400，主机发出的两种命令分别为00H和01H，含义如下：

00H——要求从机接收数据。

01H——要求从机发送数据。

从机状态字节地址为20H，状态字定义如图6-36所示。

D7	D6	D5	D4	D3	D2	D1	D0
——	——	——	——	——	——	TRDY	RRDY

从机状态字

TRDY: ”1” 从机发送状态准备好； “0” 从机发送状态未准备好。

RRDY: ”1” 从机接收状态准备好； “0” 从机接收状态未准备好。

主机程序如下（采用查询方式）：

```
#include <reg52.h>
unsigned char numbercj, numberjs, numberml, i;
unsigned char bdata state;
    sbit RRDY=state^0;
    sbit TRDY=state^1;
unsigned char trndata[16]={0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 };
unsigned char revdata[16];
main ( )
{ while(1) {
    TMOD=0X20;
    PCON=0;
    TH1=0XF4;
    TL1=0XF4;
    TR1=1;
    SCON=0XD0;
    TB8=1;
    SBUF=numbercj;           //发送从机地址
    while(TI==0); TI=0;
    while(RI==0); RI=0;
    numberjs=SBUF;
```

```
if (numberjs==numbercj) {
    TB8=0;
    numberml=SBUF;           // 发送命令
    while(TI==0); TI=0;
    while(RI==0); RI=0;      //等待从机状态反馈
        state=SBUF;
    if (numberml==0) {
        if (RRDY==1) {
            for(i=1;i<=16;i++) {
                TB8=0;
                SBUF=trndata[i];
                while(TI==0); TI=0; }
            }
        else {
            if (TRDY==1) {
                for(i=1;i<=16;i++) {
                    while (RI==0); RI=0;
                    trndata[i]=SBUF; } }
        }
    } } }
```

从机程序如下（采用中断方式）：

```
#include <reg52.h>
unsigned char numbercj, address, numberml, i;
unsigned char bdata state;
sbit RRDY=state^0;
sbit TRDY=state^1;
unsigned char trndata[16]={0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 };
unsigned char revdata[16];
main ( )
{
    TMOD=0X20;
    PCON=0;
    TH1=0XF4;
    TL1= 0XF4;
    TR1=1;
    SCON=0XF0;
    while(1);
}
```

```
uart ( ) interrupt 4 using 1
```

```
{
    if (RI==1) {RI=0;
        if (RB8==1) {
            numbercj = SBUF;
            if (numbercj ==
address)
                { SBUF =
numbercj ;
                    //反馈从机地址
                    SM2 = 0;
                    state = 0;} }
        else {if (RRDY == 0) {
            numberml = SBUF;
            if(numberml == 0) {
                RRDY = 1;
                TRDY = 0;
                SBUF = state; }
            else{TRDY = 1;
                RRDY = 0;
                SBUF = state;
            }
        }
    }
}
```

```
        else {for(i=1;i<=16;i++)
            {while (RI==0); RI=0;
                trndata[i]=SBUF; }
            SM2=1;
            RRDY=0; }
        }
    }
else { TI=0;
    if (TRDY==1) {
        for(i=1;i<=16;i++)
        {
            TB8=0;
            SBUF=trndata[i];
            while(TI==0); TI=0; }
        SM2=1;
        TRDY=0;}
    }
}
```



本章小结